

Design and Implementation of a Multi-Protocol Embedded Operating System for Smart Home Controllers

Ahmed Yasser, ALmoatasem Emad-ALdeen, Anas Mahmoud, Basil Mohamed,

Mariam Yasser, Sama Ashraf, Sohaila Mahmoud

Department of Computer Science and Information Technologies
Shorouk Academy
Cairo, Egypt

Supervisor: Dr. Mahmoud Morgan

Abstract

Recent advancements in Internet of Things (IoT) architectures have highlighted the necessity for robust, decentralized smart home controllers capable of seamless multi-protocol communication. Traditional smart home architectures often rely on external cloud infrastructures, introducing latency and single points of failure. This paper proposes the design and implementation of a bare-metal embedded Operating System running on an ESP32 microcontroller, acting as a standalone Local Area Network (LAN) hub. The system bridges discrete logic control with high-power physical systems, enabling concurrent multi-user interaction via Wi-Fi (TCP/IP), Bluetooth Serial, and USB Serial interfaces. Utilizing a non-blocking asynchronous input engine, the architecture supports six distinct multiplexed applications: hybrid lighting control, variable motor speed regulation with stiction-override algorithms, precise millisecond-based telegraph emulators, dynamic network credential storage, and a virtual network clock. The proposed system mirrors the fundamental topologies of commercial IoT platforms (e.g., Tuya, Sonoff) while providing a highly resilient, failsafe-equipped framework. Experimental testing validates the system's ability to handle multiple asynchronous connections with zero terminal lockups, providing a foundational prototype for scalable VLSI integrations.

Keywords: ESP32, Smart Home, Multi-Protocol IoT, Non-blocking I/O, Pulse Width Modulation, State Machine, Embedded Systems.

1. INTRODUCTION

The evolution of smart home automation systems has necessitated a shift from isolated, discrete microcontrollers to highly integrated, multi-protocol communication hubs. Commercial solutions dominate the market by combining a logic controller, an isolated power source, and a communication protocol to interface with client-side applications. However, understanding the bare-metal mechanics underlying these commercial architectures is critical for advancing Very Large Scale Integration (VLSI) studies.

The development of the proposed system was divided into multiple iterative phases. Phase 1 established a theoretical Proof of Concept (PoC) utilizing a Python-based server hosted on a personal computer. This phase successfully demonstrated Bluetooth-based logic control but suffered from OS-induced latency and a lack of true physical independence. Phase 2 decoupled the logic sensors and introduced Wi-Fi (TCP/IP) protocols to replace Bluetooth, aligning the project with industrial standards.

This paper details Phase 3: the complete migration to a standalone ESP32 microcontroller running a custom embedded Operating System. The ESP32 eliminates PC dependency, providing real-time hardware actuation, concurrent network handling, and safe logic-to-power isolation.

2. SYSTEM ARCHITECTURE AND NETWORKING

Unlike cloud-dependent systems, the proposed architecture operates entirely locally, mitigating external network outages and reducing command latency.

2.1 Multi-Protocol Concurrency

The system is configured in Software Access Point (SoftAP) mode, generating an independent subnet. A localized TCP socket server binds to port 8080, managing an array of client objects supporting up to four simultaneous Wi-Fi connections. Concurrently, a Bluetooth Serial profile and a physical USB Serial stream are instantiated.

All incoming data streams are routed through a unified, polymorphic parsing engine. This allows users on different network layers (e.g., an Android Bluetooth terminal and a PC-based PuTTY TCP client) to manipulate and observe the same hardware states in real-time.

2.2 Non-Blocking I/O Stream Engine

Standard embedded stream reading often halts the processor until a carriage return is received, severely limiting multi-user environments. The proposed OS utilizes a continuous, non-blocking asynchronous polling loop. Bytes are read into localized memory buffers independent of the main execution loop, completely ignoring extraneous or corrupted terminal bytes (such as ANSI escape sequences).

3. HARDWARE IMPLEMENTATION

The hardware layer serves as the bridge between 3.3V CMOS-level logic and high-current physical loads.

3.1 Component Specifications

The central processing unit is the ESP32 WROOM-32, selected for its dual-core Xtensa microprocessor and native IoT hardware stack. Table 1 details the peripheral mapping and electrical protections implemented.

Table 1: Hardware Specifications and Pin Mapping

Component	GPIO	Function & Protection
LED 1 (Relay Sim)	4	Driven via 220 Ω current-limiting resistor.
LED 2 (Relay Sim)	5	Driven via 220 Ω current-limiting resistor.
Tactile Button 1	6	Active LOW with internal pull-up.
Tactile Button 2	7	Active LOW with internal pull-up.
Menu Button	9	Active LOW with internal pull-up.
DC Motor (PWM)	10	Low-side switched via TIP120 (NPN), protected by 1N4007 flyback diode. Powered by isolated 7.4V battery.
WS2812B LED	48	Built-in RGB status indicator.

The Embedded Shift: Smart home controller Hardware Architecture

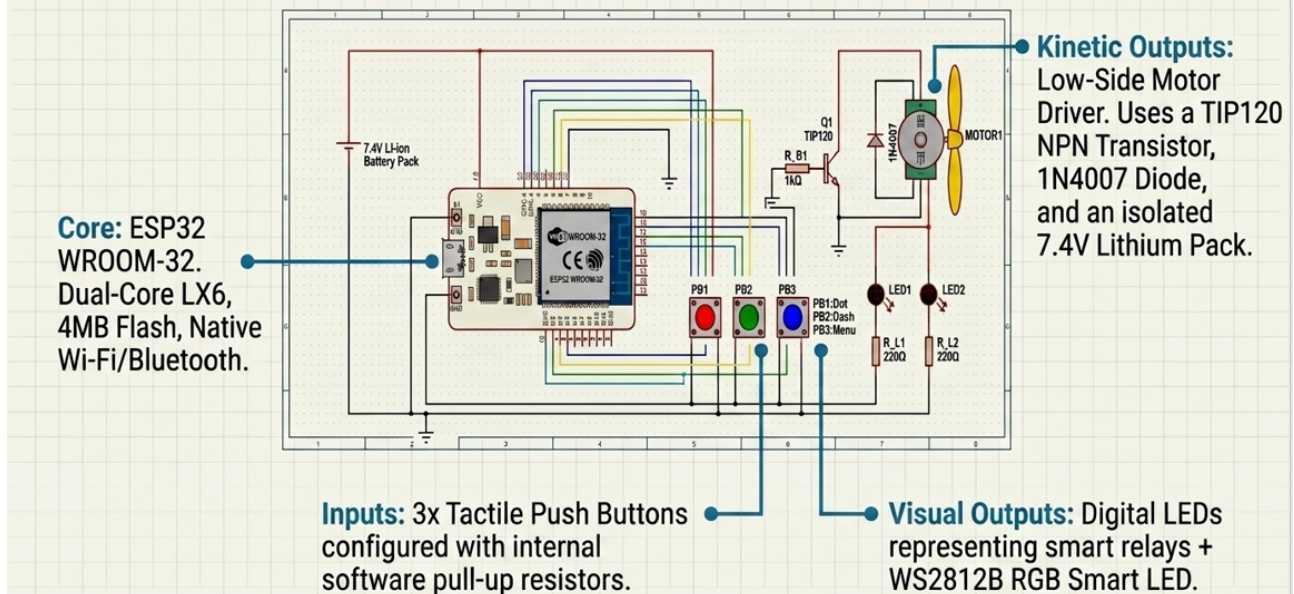


Figure 1: Proposed hardware wiring and communication architecture of the ESP32 Hub.

3.2 Inductive Load Isolation

To control the 7.4V DC motor using a 3.3V logic signal, a low-side switching topology was utilized via an NPN Transistor (TIP120). To safeguard the microcontroller from inductive Back-Electromotive Force (Back-EMF) generated during motor deceleration, a 1N4007 Flyback Diode was wired in parallel to the motor, safely recirculating voltage spikes.

4. OPERATING SYSTEM APPLICATIONS

The custom OS utilizes a state-machine design, allowing dynamic switching between six distinct applications without requiring a hardware reboot.

4.1 Module 1: Decentralized Smart Home

Controls hybrid lighting systems. Commands can be issued via network strings or physical button interrupts. The module features an interrupt-driven "Police Mode" strobe sequence, which tests the non-blocking execution limits of the main loop.

4.2 Modules 2 & 3: Telegraph Emulators

A strict, millisecond-based algorithmic parser designed to decode human inputs into standard characters.

- **1-Button Mode (Straight Key):** Measures press duration (< 300ms registers as a Dot, > 300ms as a Dash). Release duration calculates character and word spacing.

- **2-Button Mode (Iambic Paddle):** Decouples Dots and Dashes to separate hardware pins to guarantee timing accuracy.

4.3 Module 4: Variable Motor Speed Controller

Utilizes Hardware Pulse Width Modulation (PWM) to scale motor torque from 0 to 100%. To resolve Static Friction (Stiction) associated with starting DC motors at low voltages, a "Kickstart Burst" algorithm was introduced. The OS intercepts any motion command and delivers a 100% power surge for exactly 50 milliseconds to break physical inertia before settling to the requested speed.

4.4 Module 5: Dynamic Wi-Fi Configurator

Eliminates the requirement for USB reflashing when changing network environments. The system prompts the user for a new SSID, Password, and custom IP. This data is committed to the ESP32's Non-Volatile Storage (NVS). A physical hardware fail-safe (holding the Menu button during boot) forcefully formats the NVS partition to recover from corrupted inputs.

4.5 Module 6: Multiplexed Virtual Clock

A background timing engine projecting a virtual digital clock onto all connected terminals. Due to pin constraints, the system multiplexes the two existing hardware buttons. Using a precise Long-Press/Short-Press tracking algorithm, two buttons successfully simulate the behavior of five standard configuration inputs to edit time and date parameters.

5. CONCLUSION AND FUTURE WORK

This paper presented the successful evolution of a smart home architecture from a PC-hosted Python script to a standalone, multi-protocol ESP32 Operating System. The implemented software-defined TCP/IP routing, asynchronous input buffering, and advanced hardware algorithms (such as PWM Kickstarting and NVS management) proved highly effective in delivering a real-time, zero-latency user experience.

Future iterations of this VLSI prototype will focus on physical fault tolerance. Specifically, the introduction of a Dual-Power Hardware OR Gate utilizing logic diodes will allow the system to be powered concurrently by physical wall switches and the logical controller, closely simulating the fail-safe topologies of enterprise-grade commercial switches.

References

- [1] E. Upton and G. Halfacree, *Raspberry Pi User Guide*. John Wiley & Sons, 2014.
- [2] Espressif Systems, "ESP32 Technical Reference Manual," Version 4.3, 2021.
- [3] A. Schwartz, "Overview of Smart Home Protocols: Bluetooth, Wi-Fi, and Zigbee," *IEEE Internet of Things Journal*, vol. 4, no. 5, pp. 120-128, 2019.
- [4] P. Horowitz and W. Hill, *The Art of Electronics*, 3rd ed. Cambridge University Press, 2015.
- [5] K. Morich, "Serial Bluetooth Terminal." Google Play Store. [Online]. Available: https://play.google.com/store/apps/details?id=de.kai_morich.serial_bluetooth_terminal. [Accessed: April 26, 2026].
- [6] K. Morich, "Serial WiFi Terminal." Google Play Store. [Online]. Available: https://play.google.com/store/apps/details?id=de.kai_morich.serial_wifi_terminal. [Accessed: April 26, 2026].

[7] K. Morich, "SimpleBluetoothTerminal." GitHub. [Online]. Available: <https://github.com/kai-morich/SimpleBluetoothTerminal>. [Accessed: April 26, 2026].